ТЕХНИЧЕСКАЯ ИНСТРУКЦИЯ

RGSCenter

Аутентификация, авторизация и жизненный цикл browser token

Документ описывает фактическую реализацию модуля com.rgs.center.auth: структуру PostgreSQL, классы и методы Java, создание токена, cookie, восстановление SecurityContext, отзыв, истечение и подсчёт подключённых браузеров.

Проект: C:\Projects\RGSCenter
Версия документа: 1.0
Дата: 18 августа 2026

Содержание

1. Назначение и границы механизма 3

1.1. Основные компоненты . . . 3

2. Структура базы данных 3

2.1. Таблица roles . . . 3

2.2. Таблица users . . . 3

2.3. Таблица auth_sessions . . . 4

3. Последовательность входа 4

4. Классы, свойства и методы 5

4.1. Record-модели . . . 5

4.2. SQL AuthUserRepository.findByLogin . . . 5

5. Механизм создания token 5

6. Жизненный цикл token 6

6.1. Проверка каждого запроса . . . 6

6.2. Logout . . . 6

6.3. Повторный login и перезапуск приложения . . . 6

7. Cookie и CSRF 7

8. Авторизация и роли 7

9. Настройки application.properties 7

10. Эксплуатационные проверки 7

11. Карта исходных файлов 8

1. Назначение и границы механизма

RGSCenter использует собственную stateless-аутентификацию на основе случайного bearer token. Spring HTTP session не создаётся. Сырой токен хранится только в HttpOnly cookie браузера, а PostgreSQL содержит SHA-256-хэш токена и серверные признаки его действительности.

Аутентификация отвечает на вопрос, кто выполнил запрос. Авторизация в текущей версии проверяет только наличие аутентифицированного пользователя. Роль загружается и добавляется в Spring Security, но правила hasRole/hasAuthority пока не применяются.

1.1. Основные компоненты

Компонент	Ответственность
SecurityConfiguration	SecurityFilterChain, CSRF, public endpoints, stateless policy
AuthController	HTTP endpoints login, logout и connections
AuthenticationService	Проверка логина и BCrypt-пароля
AuthUserRepository	Чтение users JOIN roles
AuthSessionService	Создание, проверка, отзыв и очистка токен-сессий
AuthCookieService	Cookie RGS_AUTH_TOKEN
AuthTokenFilter	Восстановление AuthPrincipal для каждого запроса
PostgreSQL	roles, users, auth_sessions

2. Структура базы данных

Файл center/src/main/resources/schema.sql выполняется Spring Boot при запуске благодаря свойству spring.sql.init.mode=always. Команды создания и начального наполнения идемпотентны.

2.1. Таблица roles

Поле	Тип/ограничение	Назначение
id	INTEGER IDENTITY, PK	Стабильный идентификатор роли
rolename	VARCHAR(256), NOT NULL, UNIQUE	Системное имя роли

id	rolename	Authority
1	service	ROLE_SERVICE
2	administrator	ROLE_ADMINISTRATOR
3	operator	ROLE_OPERATOR

2.2. Таблица users

Поле	Тип/ограничение	Назначение
id	INTEGER IDENTITY, PK	Идентификатор пользователя
login	VARCHAR(256), NOT NULL, UNIQUE	Логин
hash	VARCHAR(256), NOT NULL	BCrypt-хэш пароля
role	INTEGER, NOT NULL, FK roles.id	Роль пользователя

2.3. Таблица auth_sessions

Поле	Тип	Назначение
id	UUID, PK	Идентификатор серверной сессии
user_id	INTEGER, FK users.id	Пользователь; ON DELETE CASCADE
token_hash	VARCHAR(64), UNIQUE	SHA-256 сырого токена в hex
created_at	TIMESTAMPTZ	Время создания
last_seen_at	TIMESTAMPTZ	Последняя подтверждённая активность
expires_at	TIMESTAMPTZ	Абсолютное окончание действия
revoked_at	TIMESTAMPTZ, NULL	Момент отзыва; NULL для действующей сессии
remote_address	VARCHAR(64)	IP клиента
user_agent	VARCHAR(512)	User-Agent браузера

Индексы: idx_auth_sessions_active по revoked_at, expires_at, last_seen_at и idx_auth_sessions_user по user_id.

3. Последовательность входа

1. Форма входа
   Браузер получает GET /login и отправляет POST /auth/login с login, password и CSRF.
2. Контроллер
   AuthController.login(...) отзывает старый токен этого браузера и вызывает AuthenticationService.login(...).
3. Чтение пользователя
   AuthUserRepository.findByLogin(login) выполняет users JOIN roles.
4. Проверка пароля
   PasswordEncoder.matches(password, passwordHash) выполняет BCrypt-проверку. Для отсутствующего login используется DUMMY_PASSWORD_HASH.
5. Создание principal
   Создаётся AuthPrincipal(id, login, roleName).
6. Создание токена
   AuthSessionService.issue(...) генерирует 32 случайных байта, сохраняет SHA-256 и возвращает IssuedToken.
7. Cookie
   AuthCookieService.write(...) отправляет RGS_AUTH_TOKEN с HttpOnly, SameSite=Strict, Path=/ и Max-Age до expiresAt.

4. Классы, свойства и методы

Класс	Свойства	Ключевые методы
SecurityConfiguration	Собственных полей нет	passwordEncoder(); securityFilterChain(...)
AuthController	authenticationService; sessionService; cookieService	loginPage(); login(); logout(); connectionStatus(); clientAddress()
AuthenticationService	DUMMY_PASSWORD_HASH; userRepository; sessionService; passwordEncoder	login(...)
AuthUserRepository	jdbcClient	findByLogin(login)
AuthSessionService	TOKEN_BYTES; jdbcClient; secureRandom; sessionDuration; activeWindow	issue(); authenticate(); revoke(); countConnected(); purgeOldSessions(); hash(); truncate()
AuthCookieService	COOKIE_NAME; secure	read(); write(); clear(); baseCookie()
AuthTokenFilter	log; cookieService; sessionService	doFilterInternal(...)
SpaCsrfTokenRequestHandler	plain; xor	handle(); resolveCsrfTokenValue()
AuthModelAdvice	Собственных полей нет	addAuthenticationDetails()

4.1. Record-модели

Record	Свойства	Назначение
AuthUser	id, login, passwordHash, roleId, roleName	Проекция users JOIN roles при password login
AuthPrincipal	id, login, roleName	Principal текущего запроса; getName() возвращает login
IssuedToken	value, expiresAt	Сырой токен и абсолютное окончание действия
AuthenticatedSession	sessionId, principal	Проверенная запись сессии и пользователь

4.2. SQL AuthUserRepository.findByLogin

```
SELECT u.id, u.login, u.hash, r.id AS role_id, r.rolename
FROM users u
JOIN roles r ON r.id = u.role
WHERE u.login = :login
```

5. Механизм создания token

Метод AuthSessionService.issue(AuthPrincipal, remoteAddress, userAgent) создаёт новый токен в одной транзакции с записью серверной сессии.

Шаг	Операция	Результат
1	new byte[TOKEN_BYTES], TOKEN_BYTES = 32	Буфер 256 бит
2	SecureRandom.nextBytes(bytes)	Криптографически случайное значение
3	Base64.getUrlEncoder().withoutPadding()	URL-safe строка, обычно 43 символа
4	MessageDigest SHA-256	64 hex-символа для token_hash
5	UUID.randomUUID()	auth_sessions.id
6	now + sessionDuration	expires_at, по умолчанию 8 часов
7	INSERT auth_sessions	Хэш и метаданные сохранены в PostgreSQL

Шаг	Операция	Результат
8	new IssuedToken(token, expiresAt)	Сырой token возвращён контроллеру

Критическое свойство: сырой token никогда не сохраняется в базе. Компрометация таблицы auth_sessions не даёт готовый bearer token для входа.

6. Жизненный цикл token

Состояние	Условие	Поведение
Создан	Успешный login	created_at = last_seen_at = now; expires_at = now + 8h
Действует	revoked_at IS NULL AND expires_at > now	AuthTokenFilter создаёт Authentication
Активен	last_seen_at >= now - 90s	Учитывается countConnected()
Отозван	revoke(token)	revoked_at = now; повторная проверка отклоняется
Истёк	expires_at <= now	Проверка отклоняется; last_seen_at не обновляется
Удалён	Истёк или отозван более 7 дней	purgeOldSessions() удаляет строку при следующем login

6.1. Проверка каждого запроса

AuthTokenFilter.doFilterInternal(...) вызывается один раз на запрос до стандартного UsernamePasswordAuthenticationFilter. Он читает cookie, вызывает AuthSessionService.authenticate(token), а затем создаёт UsernamePasswordAuthenticationToken с AuthPrincipal и authority ROLE_<ROLENAME>.

```
SELECT s.id, u.id AS user_id, u.login, r.rolename
FROM auth_sessions s
JOIN users u ON u.id = s.user_id
JOIN roles r ON r.id = u.role
WHERE s.token_hash = :tokenHash
  AND s.revoked_at IS NULL
  AND s.expires_at > :now
```

last_seen_at обновляется не чаще одного раза в 15 секунд. Это уменьшает количество UPDATE при загрузке страницы, ресурсов и AJAX-запросов. Обновление активности не продлевает expires_at.

6.2. Logout

1. Чтение cookie
   AuthController.logout() вызывает AuthCookieService.read(request).
2. Отзыв
   AuthSessionService.revoke(token) устанавливает revoked_at = now().
3. Удаление cookie
   AuthCookieService.clear(response) отправляет пустое значение с Max-Age=0.
4. Очистка контекста
   SecurityContextHolder.clearContext() и redirect /login.

6.3. Повторный login и перезапуск приложения

- Перед выдачей нового токена AuthController.login() отзывает старый токен текущего браузера.
- Токен не ротируется автоматически во время 8-часового периода.
- Перезапуск Java-приложения не аннулирует токены: состояние хранится в PostgreSQL.
- Удаление users удаляет связанные auth_sessions через ON DELETE CASCADE.

7. Cookie и CSRF

Cookie	Назначение	Доступ JavaScript	Параметры
RGS_AUTH_TOKEN	Аутентификация	Нет: HttpOnly=true	SameSite=Strict; Path=/; Secure из настройки
XSRF-TOKEN	Защита изменяющих запросов	Да	Читается JS и передаётся X-XSRF-TOKEN

SpaCsrfTokenRequestHandler поддерживает маскированное значение CSRF в HTML-форме и сырое значение из заголовка X-XSRF-TOKEN. CSRF token не идентифицирует пользователя и не заменяет RGS_AUTH_TOKEN.

Endpoint /auth/logout исключён из CSRF-проверки. Он может только отозвать текущий SameSite=Strict token и удалить cookie, поэтому logout остаётся работоспособным из старой вкладки.

8. Авторизация и роли

AuthTokenFilter преобразует roles.rolename в authority верхнего регистра с префиксом ROLE_. Например, administrator становится ROLE_ADMINISTRATOR.

Текущая конфигурация использует .anyRequest().authenticated(). Поэтому service, administrator и operator имеют одинаковый доступ ко всем защищённым endpoint. Для разграничения можно заменить общую проверку правилами hasRole(...).

```
.requestMatchers("/admin/**").hasRole("ADMINISTRATOR")
.requestMatchers("/service/**").hasRole("SERVICE")
.anyRequest().hasAnyRole("SERVICE", "ADMINISTRATOR", "OPERATOR")
```

9. Настройки application.properties

Свойство	Значение	Назначение
spring.datasource.url	jdbc:postgresql://localhost:5432/rgscenter	PostgreSQL
spring.sql.init.mode	always	Выполнять schema.sql при запуске
rgs.auth.session-duration	8h	Абсолютный срок token
rgs.auth.active-window	90s	Окно countConnected()
rgs.auth.cookie-secure	env RGSCENTER_COOKIE_SECURE, default false	Secure для HTTPS

Для production через HTTPS необходимо установить RGSCENTER_COOKIE_SECURE=true. Учетные данные БД следует передавать через RGSCENTER_DB_URL, RGSCENTER_DB_USER и RGSCENTER_DB_PASSWORD, а не хранить production secrets в репозитории.

10. Эксплуатационные проверки

1. Проверка
   После login в ответе присутствует Set-Cookie RGS_AUTH_TOKEN с HttpOnly и SameSite=Strict.
2. Проверка
   В auth_sessions хранится token_hash длиной 64 символа, а не значение cookie.
3. Проверка
   После logout соответствующая строка получает revoked_at, а cookie удаляется Max-Age=0.

4 Проверка
После expires_at токен отклоняется независимо от last_seen_at.

5 Проверка
Два разных токена, активных за последние 90 секунд, дают warning=true в /auth/connections.

6 Проверка
В production cookie передаётся только по HTTPS при RGSCENTER_COOKIE_SECURE=true.

7 Проверка
Пароли service/admin/operator после установки заменены на уникальные безопасные значения.

11. Карта исходных файлов

Файл	Содержание
center/src/main/java/com/rgs/center/auth/SecurityConfiguration.java	Spring Security filter chain
center/src/main/java/com/rgs/center/auth/AuthController.java	Login/logout/connections endpoints
center/src/main/java/com/rgs/center/auth/AuthenticationService.java	Проверка credentials
center/src/main/java/com/rgs/center/auth/AuthUserRepository.java	users JOIN roles
center/src/main/java/com/rgs/center/auth/AuthSessionService.java	Token lifecycle
center/src/main/java/com/rgs/center/auth/AuthCookieService.java	RGS_AUTH_TOKEN cookie
center/src/main/java/com/rgs/center/auth/AuthTokenFilter.java	SecurityContext на каждый request
center/src/main/java/com/rgs/center/auth/SpaCsrfTokenRequestHandler.java	HTML/JS CSRF
center/src/main/resources/schema.sql	roles, users, auth_sessions
center/src/main/resources/application.properties	DB и auth settings
center/src/main/resources/static/js/auth-status.js	Опрос /auth/connections каждые 30 секунд